LangChain vs LangGraph

LangChain building blocks | **LangGraph** stateful orchestration | Together = production agents

1. The core difference

LangChain	LangGraph
Composable building blocks	Stateful workflow orchestrator
Input -> LLM -> Output (linear)	Nodes + Edges + Loops (graph)
Prompts, chains, tools, memory	State, nodes, edges, checkpointers
Stateless pipeline	Stateful across turns and steps
No loops or branching	Conditionals, cycles, multi-agent

Mental model: LangChain is the LEGO bricks. LangGraph is the blueprint that decides which brick goes where, in what order, and can loop back.

2. LangChain

What it does

Ready-made classes for models, prompts, parsers, tools, and memory. Compose them with the | pipe operator (LCEL) to build a chain.

Build a chain — step by step

1. Import ChatOpenAI, ChatPromptTemplate, StrOutputParser
2. Write a prompt template with {variable} placeholders
3. Chain them: prompt | llm | parser
4. Run with .invoke({variable: value})

```
from langchain_openai import ChatOpenAI
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.output_parsers import StrOutputParser
```

```
llm    = ChatOpenAI(model="gpt-4o", temperature=0)
prompt = ChatPromptTemplate.from_messages([
    ("system", "You are a helpful assistant."),
    ("human",  "Summarise in one line: {text}")
])
chain  = prompt | llm | StrOutputParser()   # LCEL pipe
result = chain.invoke({"text": "LangGraph tutorial..."})
```

Key classes

Class/method	What it does
`ChatOpenAI`	Wraps any chat model. Set model, temperature, streaming.
`ChatPromptTemplate`	Structured prompt with system + human messages and {vars}.
`StrOutputParser`	Turns an AIMessage into a plain string.
`@tool decorator`	Makes any Python function callable by the LLM. Docstring = description.
`llm.bind_tools(tools)`	Attaches tools so the LLM knows it can call them.
`RunnableLambda`	Wraps any Python function as a chain step.
`ConversationBufferMemory`	Verbatim history. Simple. Use for short sessions.
`ConversationSummaryMemory`	Compresses old messages via LLM. Use for long sessions.

When to use LangChain alone

- Single-shot Q&A or classification
- RAG: retrieve docs -> prompt -> answer
- Document summarisation or extraction
- Any linear flow with no loops

LangChain flow (always linear)

Prompt	LLM	Parser	Output

Each step passes its output to the next. No loops, no shared state, no branching.

3. LangGraph

What it does

Build stateful graphs where each node is a Python function that reads and writes shared state. Supports loops, conditional branching, and memory across turns.

Build a graph — step by step

5. Define a TypedDict as your state schema
6. Write node functions: take state -> return dict of updates
7. Create a StateGraph(YourState)
8. Register nodes with graph.add_node(name, fn)
9. Wire them with add_edge() and add_conditional_edges()
10. Compile: app = graph.compile()
11. Run: app.invoke({initial state dict})

```
from langgraph.graph import StateGraph, START, END
from typing import TypedDict, Annotated
import operator

class State(TypedDict):
    messages: Annotated[list, operator.add]  # append, not replace
    done: bool

def call_llm(state):
    response = llm.invoke(state["messages"])
    return {"messages": [response]}

def router(state) -> str:
    return "end" if state["done"] else "call_llm"

graph = StateGraph(State)
graph.add_node("call_llm", call_llm)
graph.add_edge(START, "call_llm")
graph.add_conditional_edges("call_llm", router, {"end": END, "call_llm":
"call_llm"})
app = graph.compile()
```

Key concepts

Class/method	What it does
`StateGraph(State)`	Creates a graph tied to your state schema.
`add_node(name, fn)`	Registers a node. fn reads state, returns partial update.
`add_edge(a, b)`	Fixed edge: always go from a to b.
`add_conditional_edges`	Dynamic: calls a router fn to pick the next node.
`Annotated[list, add]`	State field that appends on update instead of replacing.
`ToolNode(tools)`	Pre-built node that executes tool_calls from an LLM message.
`compile(checkpointer=...)`	Locks the graph. Attach MemorySaver for persistent memory.
`thread_id in config`	Separates conversations. Same ID = same session memory.
`interrupt_before=[...]`	Pause before a node — human-in-the-loop approval.

When to use LangGraph

- Multi-turn chatbots with memory across sessions
- ReAct agents: think -> call tool -> observe -> repeat
- Multi-agent systems: supervisor routing to sub-agents
- Human-in-the-loop: pause and wait for approval
- Any flow that needs loops, retries, or branching

LangGraph ReAct agent flow (with loop)

START	Planner	Researcher	Tool runner	Synth	END

After Tool runner, graph loops back to Researcher until no tool calls remain — then goes to Synth and END.

4. How they work together

LangChain lives inside LangGraph nodes. A node is just a Python function — inside it you can use any LangChain chain. LangGraph only sees the dict you return.

```
# LangChain chain inside a LangGraph node
chain = prompt | llm | StrOutputParser()    # <- LangChain

def synthesizer_node(state):                # <- LangGraph node
    report = chain.invoke({"messages": state["messages"]})
    return {"final_report": report}         # partial state update
```

LangChain provides	LangGraph provides
• @tool — callable tools • ChatPromptTemplate — prompts • llm.bind_tools() — tool binding • LCEL pipe — compose steps • Output parsers — clean responses	• StateGraph — shared state • Nodes — call any LC chain • Edges — control flow + loops • ToolNode — execute tool calls • Checkpointer — persistence

5. Key things to remember

1. Pipe operator = chain prompt | llm | parser. Left output feeds right input. Each component must implement .invoke().

2. Node = function def my_node(state) -> dict. Reads state, returns partial update. LangGraph merges it.

3. Annotated[list, operator.add] Without this, each node overwrites the messages list. With it, updates are appended.

4. Conditional edge needs a router fn add_conditional_edges("node", router_fn, {"key": "next_node"}). Router returns a string key.

5. Always compile() before invoke() StateGraph is a blueprint. graph.compile() returns the runnable app.

6. thread_id scopes memory Same thread_id = same conversation memory. Each user session needs its own thread_id.

6. FAQ

Q: Can I use LangChain without LangGraph?

Yes. For simple pipelines — RAG, summarisation, extraction — chains alone are enough. Add LangGraph only when you need loops, persistent state, or multi-agent orchestration.

Q: What is LCEL?

LangChain Expression Language. The | pipe syntax that composes Runnables. Every component (prompt, LLM, parser, retriever) is a Runnable, so they all plug together the same way.

Q: add_edge vs add_conditional_edges?

add_edge is fixed — always A to B. add_conditional_edges calls a router function and picks the next node based on what it returns.

Q: When does the ReAct loop stop?

When the router sees no tool_calls in the last message. LLM returns a tool call (loop continues) or plain text (loop ends, move to synthesizer).

Q: What is ToolNode?

A pre-built LangGraph node that reads tool_calls from the last AIMessage, executes each tool, and returns ToolMessages. No boilerplate needed.

Q: Two nodes update the same state key — what happens?

For plain keys (str, bool), last write wins. For Annotated[list, operator.add], updates are appended. The reducer controls merge behaviour.

Q: Buffer vs Summary memory?

Buffer: keeps all messages verbatim — simple but grows each turn. Summary: compresses old messages via an LLM call — compact but costs extra tokens. Buffer for short sessions, Summary for long ones.

Q: What is create_react_agent?

Shortcut from langgraph.prebuilt. Builds the full ReAct loop (researcher + ToolNode + router) in one call: create_react_agent(model=llm, tools=[...]).

7. One-page cheatsheet

Task	Use this
Q&A / chat	`LangChain — prompt | llm | parser`
RAG pipeline	`LangChain — retriever | prompt | llm | parser`
Tool-using agent	`LangGraph — create_react_agent(llm, tools)`
Multi-turn chat	`LangGraph — compile(checkpointer=MemorySaver())`
Custom agent loop	`LangGraph — StateGraph + add_conditional_edges`
Multi-agent system	`LangGraph — supervisor node + sub-agent nodes`
Human approval	`LangGraph — compile(interrupt_before=["node"])`
Long chat memory	`LangChain — ConversationSummaryMemory(llm=llm)`
Last N messages	`LangChain — ConversationBufferWindowMemory(k=5)`

Remember: LangChain builds the pieces. LangGraph decides when each piece runs, what state it sees, and where to go next. In production, you almost always need both.